Qué es?

En todos los lenguajes hay un concepto muy importante llamado `scope`, o *alcance* en español.

El `scope` define qué tanto *alcance* tiene una variable en otras partes del código.

Las funciones tiene su propio `scope`. A este le llamamos `scope local`.

Esto significa que si yo defino una variable dentro de la función, esta solo va a existir **dentro** de ese `scope local`. Dentro de ese **alcance**.

Apenas termine la ultima de instrucción de la función, **todas sus variables** dejarán de existir.

No serán accesibles desde otras funciones o desde afuera.

Consideremos el siguiente código:

Copiar

```
def declare_variable():
  variable_inside_function_scope = 8
  print(f'Inside function: {variable_inside_function_scope}')


declare_variable()
print(f'Out of function: {variable_inside_function_scope}')
```

```python
def declare_variable():
    variable_inside_function_scope = 8
    print(f'Inside function: {variable_inside_function_scope}')


declare_variable()
print(f'Out of function: {variable_inside_function_scope}')
```

A primera vista podemos notar que VSCode nos está indicando que hay algo mal con el segundo print, pero intentemos ejecutarlo para ver qué pasa.

```
Inside function: 8
Traceback (most recent call last):
File "/Users/alekcastillo/Documents/repos/d0ap/Secciones/Funciones/outside_scope.py", line 7, in <module>
print(f'Out of function: {variable_inside_function_scope}')
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
NameError: name 'variable_inside_function_scope' is not defined
```

Podemos ver que la variable si se pudo printear una vez. Esto gracias a la línea 3 ya que el `print` estaba dentro de la función. Es decir, la variable aun existía en el scope.

Sin embargo, al intentar printearla fuera de la función, nos tiró el error de que la variable no está definida. Esto debido a que ya estamos fuera de su **scope**.

Las variables solo existen dentro de su scope. En este caso, dentro de la función `declare_variable`. Una vez que salimos de ahi, esa variable ya no existe.

Scope Global

La jerarquía de los scopes va de afuera hacia adentro.

Es decir que las variables declaradas dentro de una función no son accesibles desde afuera, pero las declaradas desde afuera sí son accesibles desde adentro.

A este alcance le llamamos el `scope global`, y contiene todas las variables declaradas **fuera** de funciones.

Consideremos el siguiente código:

```
1  variable_outside_function_scope = 7
2
3  def print_variable():
4    print(f'Inside function: {variable_outside_function_scope}')
5
6
7  print_variable()
8  print(f'Outside function: {variable_outside_function_scope}')
```

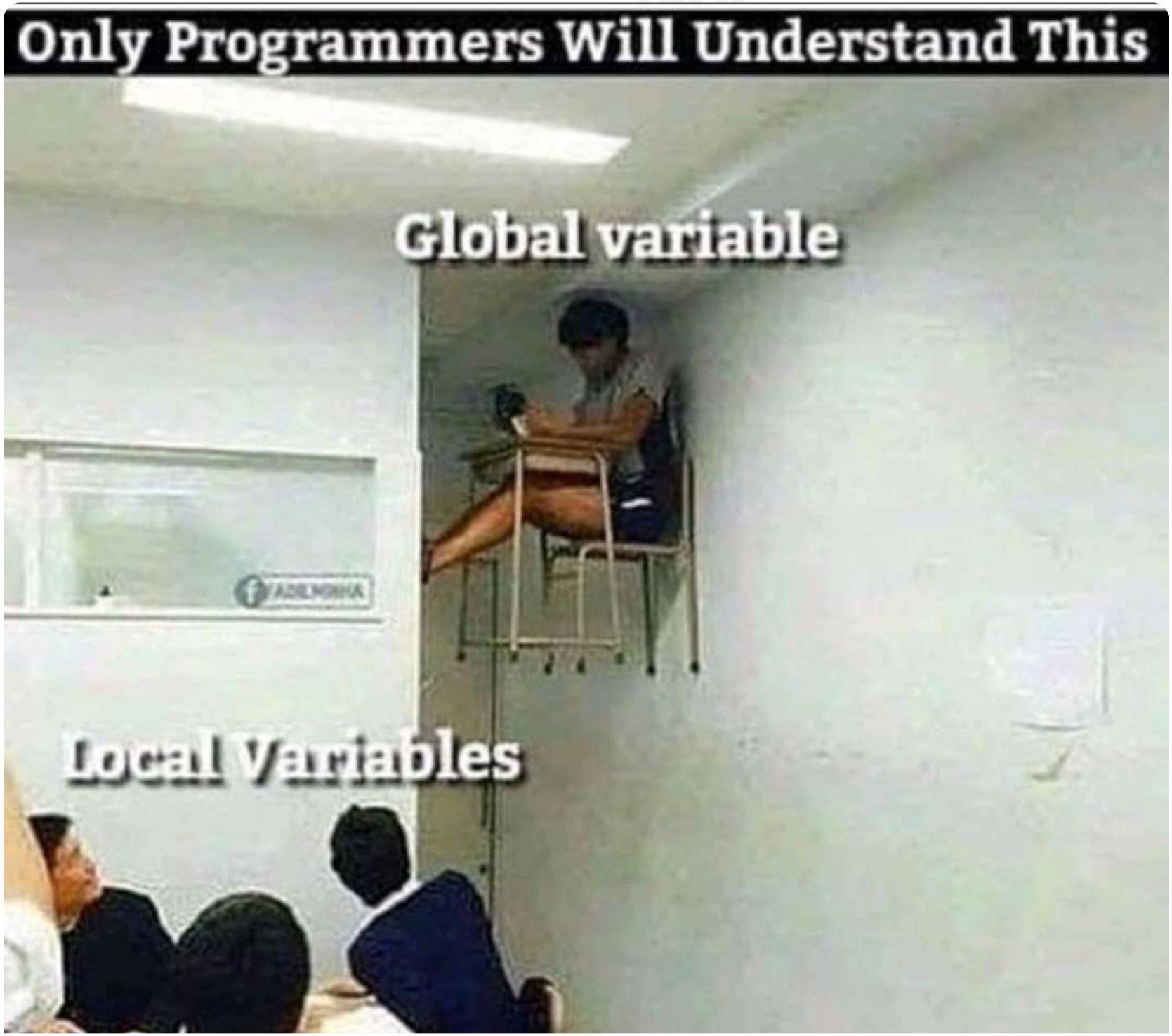
Copiar

Si ejecutamos este código, veremos que no ningún hay problema:

Inside function: 7
Outside function: 7

La función es capaz de accesar la variable ya que esta es parte del `scope global`.

Por esto se le llama una `variable global`.


Only Programmers Will Understand This
Global variable
Local Variables



Sin embargo, utilizar variables globales **no es** una práctica recomendada...

Variables globales

Las `variables globales` son armas de doble filo.

Son útiles cuando tenemos valores constantes (*que no cambian*) que necesitamos accesar en distintas partes del código.

> **Sin embargo, nunca hay que usarlas para guardar valores mutables (que pueden cambiar).**

Usarlas para valores cambiantes hace que sea muy difícil seguir el flujo de cambios en ellas, y en general, lleva a malas prácticas de programación.

> Por ejemplo, si yo tengo una variable como `numbers_list` y 7 funciones distintas que la modifican, se me va a dificultar encontrar dónde y por qué fue que *x* cambio sucedió.

Aquí algunos artículos que hablan más sobre este tema:

ericnormand.me

Global Mutable State

Global mutable state is one of the biggest drivers of complexity in software systems. We tackle a definition and how to reduce our reliance on it.

Open link

www.baeldung.com

Why Is Using Global Variables Considered a Bad Practice? | Baeldung on Computer Science

Learn about global variables.

Open link

De variables globales a variables locales

Usar `variables globales` no es recomendado - siempre debemos usar `variables locales` (declaradas dentro de funciones) y pasarlas de función a función usando `parámetros` y `returns`.

Veamos este ejemplo donde tenemos una lista global que es referenciada en varias funciones.

Copiar

```
numbers_list = [53, 60, 32, 62, 400, 10]


def remove_tenths():
  index = 0
  while (index < len(numbers_list)):
    if numbers_list[index] % 10 == 0:
      numbers_list.pop(index)
    else:
      index += 1


def multiply_numbers_by_2():
  for index, number in enumerate(numbers_list):
    numbers_list[index] = number * 2


def main():
  remove_tenths()
  multiply_numbers_by_2()
  print(numbers_list)


if __name__ == "__main__":
  main()
```

La manera correcta de arreglar este común problema es convertirla en una variable local dentro del entrypoint y pasarla al resto de funciones como un parámetro.

Copiar

```python
def remove_tenths(numbers_list):
  index = 0
  while (index < len(numbers_list)):
    if numbers_list[index] % 10 == 0:
      numbers_list.pop(index)
    else:
      index += 1


def multiply_numbers_by_2(numbers_list):
  for index, number in enumerate(numbers_list):
    numbers_list[index] = number * 2


def main():
  numbers_list = [53, 60, 32, 62, 400, 10]
  remove_tenths(numbers_list)
  multiply_numbers_by_2(numbers_list)
  print(numbers_list)


if __name__ == "__main__":
  main()
```

Puntos Importantes

El `scope` se refiere al *alcance* que tienen las variables.

Las funciones tienen su propio scope: el `scope local`.

Las variables declaradas dentro de este scope se llaman `variables locales`, y dejarán de existir apenas termine la ejecución de su función.

También existe un `scope global` que tiene todo lo declarado fuera de funciones.

Las variables declaradas dentro de este scope se llaman `variables globales`.

Las `variables globales` nunca deben ser usadas para datos mutables.

Lo mejor es usar `variables locales` y pasarlas de función a función usando `parámetros` y `returns`.